Lab 2: Building & Deploying the API Layer

From a Blank Folder to a Running .NET API + PostgreSQL on Kubernetes

What You Will Build

By the end of this lab, you will have a PostgreSQL database and a .NET Core Web API running as separate Pods inside your local Kubernetes cluster, communicating with each other using DNS-based service discovery, with the database password injected as a Secret and all other configuration injected as a ConfigMap.

Learning Objectives

- Containerize a .NET Core Web API using a multi-stage Dockerfile
- Separate sensitive configuration (Secrets) from non-sensitive configuration (ConfigMaps)
- Inject both into a running container as environment variables
- Understand and prove DNS-based service discovery between two Pods
- Configure liveness and readiness probes correctly

Prerequisites

- .NET 8 SDK installed (dotnet --version shows 8.x)
- Docker Desktop running, with Kubernetes enabled
- kubectl get nodes shows a Ready node
- Lab 1 cluster cleared: kubectl get pods returns no application pods

Part A — Create the .NET API Project

Step A1 — Create a clean project folder

Use a brand-new folder. Do not reuse a Visual Studio-generated project for this lab — Visual Studio's own container tooling builds debug-only images that behave differently from a plain docker build, and mixing the two causes confusing port and entrypoint issues.

```
mkdir C:\k8s-training\MyApi
cd C:\k8s-training\MyApi
```

```
Last login: Wed Jun 24 11:39:38 on ttys000
/dev/fd/12:25: command not found: compdef
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 44 June 25 % mkdir k8s-training
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 44 June 25 % cd k8s-training
vaibhavgupta@FVFG905WQ05F-vaibhavgupta k8s-training %
```

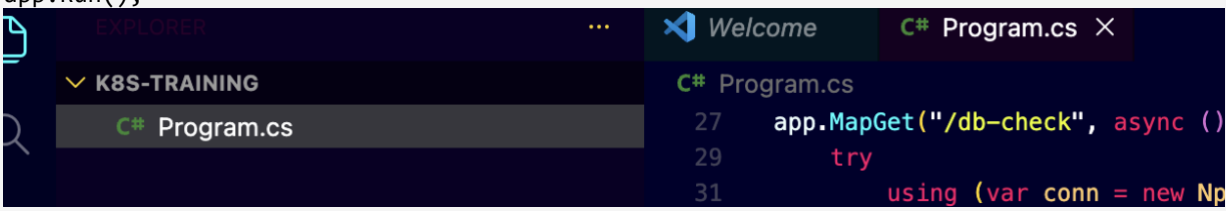
Step A2 — Create Program.cs

Create a file named Program.cs in this folder with the following content. This is a minimal API with four endpoints: a root health check, a dedicated /health endpoint, a /db-check endpoint that proves Postgres connectivity, and a /config endpoint that echoes back what configuration the container received.

Program.cs

```
using Npgsql;

var builder = WebApplication.CreateBuilder(args);

// Read DB connection details from environment variables
var dbHost = Environment.GetEnvironmentVariable("DB_HOST") ?? "localhost";
var dbPort = Environment.GetEnvironmentVariable("DB_PORT") ?? "5432";
var dbName = Environment.GetEnvironmentVariable("DB_NAME") ?? "appdb";
var dbUser = Environment.GetEnvironmentVariable("DB_USER") ?? "appuser";
var dbPassword = Environment.GetEnvironmentVariable("DB_PASSWORD") ?? "";

var connectionString =

$"Host={dbHost};Port={dbPort};Database={dbName};Username={dbUser};Password={dbPasswor
d}";

var app = builder.Build();

app.MapGet("/health", () =>
{
    return Results.Ok(new { status = "healthy", timestamp = DateTime.UtcNow });
});

app.MapGet("/", () =>
{
    return Results.Ok(new { message = "API is running", version = "1.0" });
});

app.MapGet("/db-check", async () =>
{
    try
    {
        using (var conn = new NpgsqlConnection(connectionString))
        {
            await conn.OpenAsync();
            using (var cmd = conn.CreateCommand())
            {
```

```
                cmd.CommandText = "SELECT version()";
                var result = await cmd.ExecuteScalarAsync();
                return Results.Ok(new
                {
                    status = "connected",
                    database = dbName,
                    host = dbHost,
                    postgresVersion = result?.ToString() ?? "unknown"
                });
            }
        }
    }
    catch (Exception ex)
    {
        return Results.Problem(
            detail: ex.Message,
            title: "Database Connection Failed",
            statusCode: 503
        );
    }
});

app.MapGet("/config", () =>
{
    return Results.Ok(new
    {
        db_host = dbHost,
        db_port = dbPort,
        db_name = dbName,
        db_user = dbUser,
        api_port = Environment.GetEnvironmentVariable("API_PORT") ?? "8080"
    });
});

app.Run();
```



Step A3 — Create MyApi.csproj

MyApi.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
```

```
  <ItemGroup>
    <PackageReference Include="Npgsql" Version="8.0.0" />
  </ItemGroup>

</Project>
```

Step A4 — Create the Dockerfile

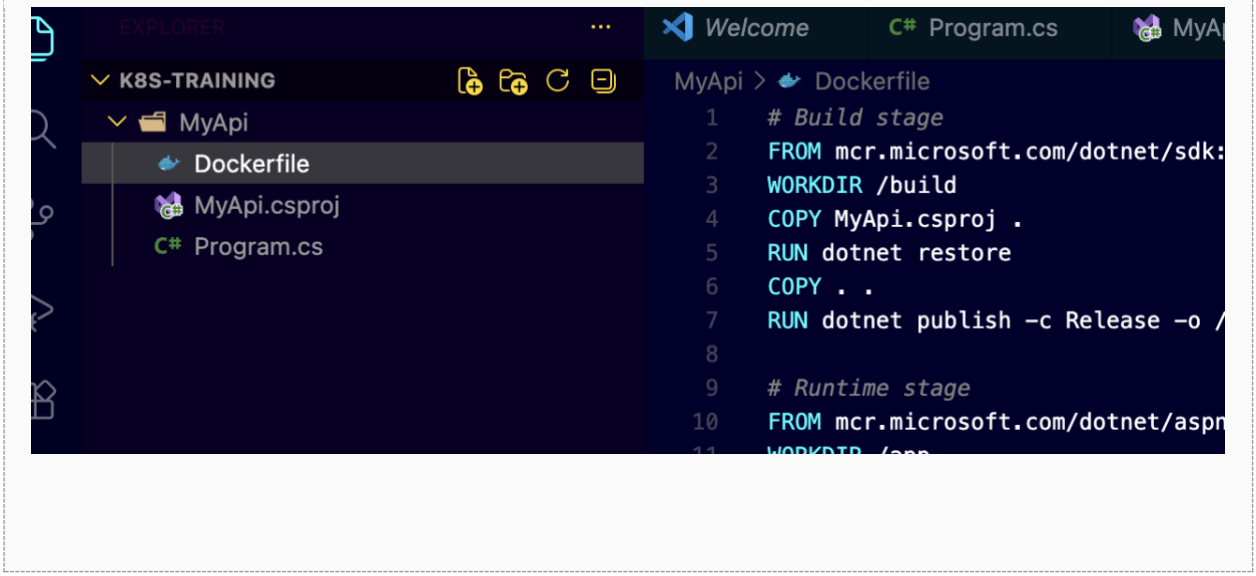
This is a multi-stage build: the first stage compiles the application using the full SDK image, the second stage copies only the compiled output into a much smaller runtime-only image. This keeps the final image lean.

Dockerfile

```
# Build stage
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /build
COPY MyApi.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app

# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app .
EXPOSE 8080
ENTRYPOINT ["dotnet", "MyApi.dll"]
```

Confirm your folder now contains exactly these three files:

```
C:\k8s-training\MyApi\
  |-- Program.cs
  |-- MyApi.csproj
  \-- Dockerfile
```



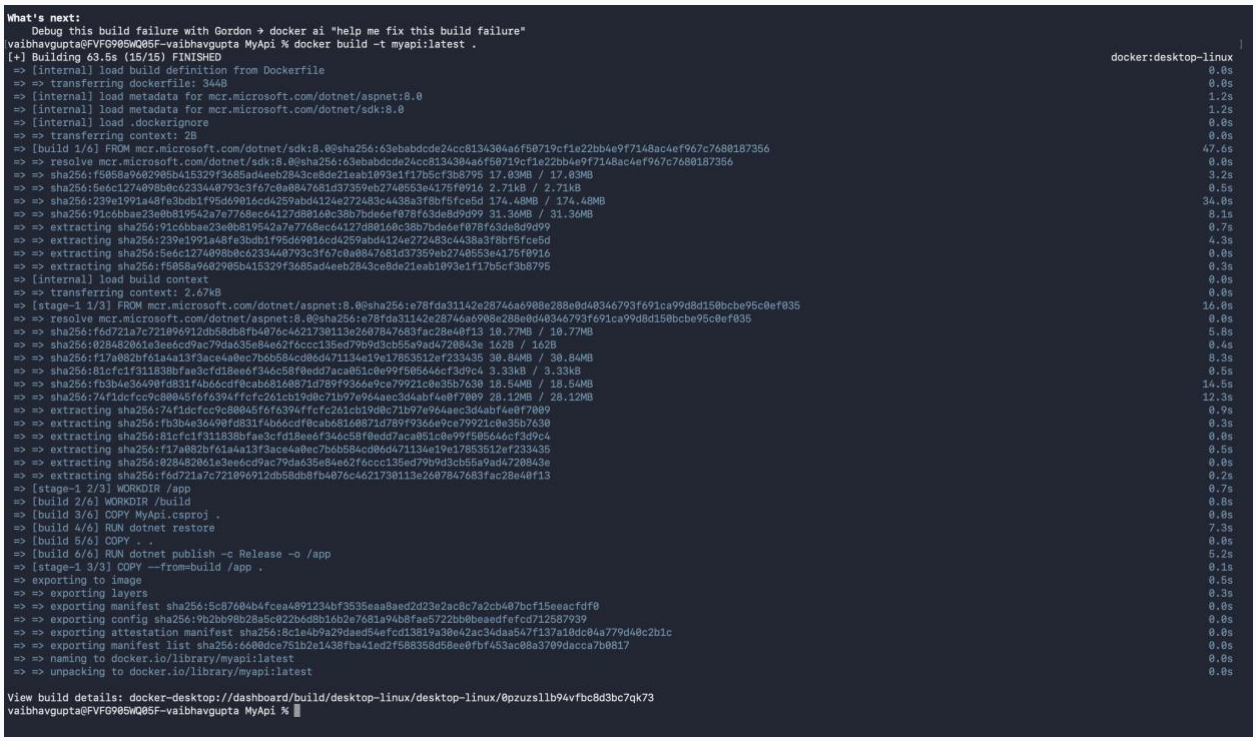
Part B — Build and Verify the Container Image

Step B1 — Build the image

Run this from inside the MyApi folder, in a plain terminal (Command Prompt or PowerShell — not a Visual Studio integrated terminal).

```
docker build -t myapi:latest .
```

Wait for the build to complete with a message ending in Successfully tagged myapi:latest. The first build takes a few minutes while base images download.



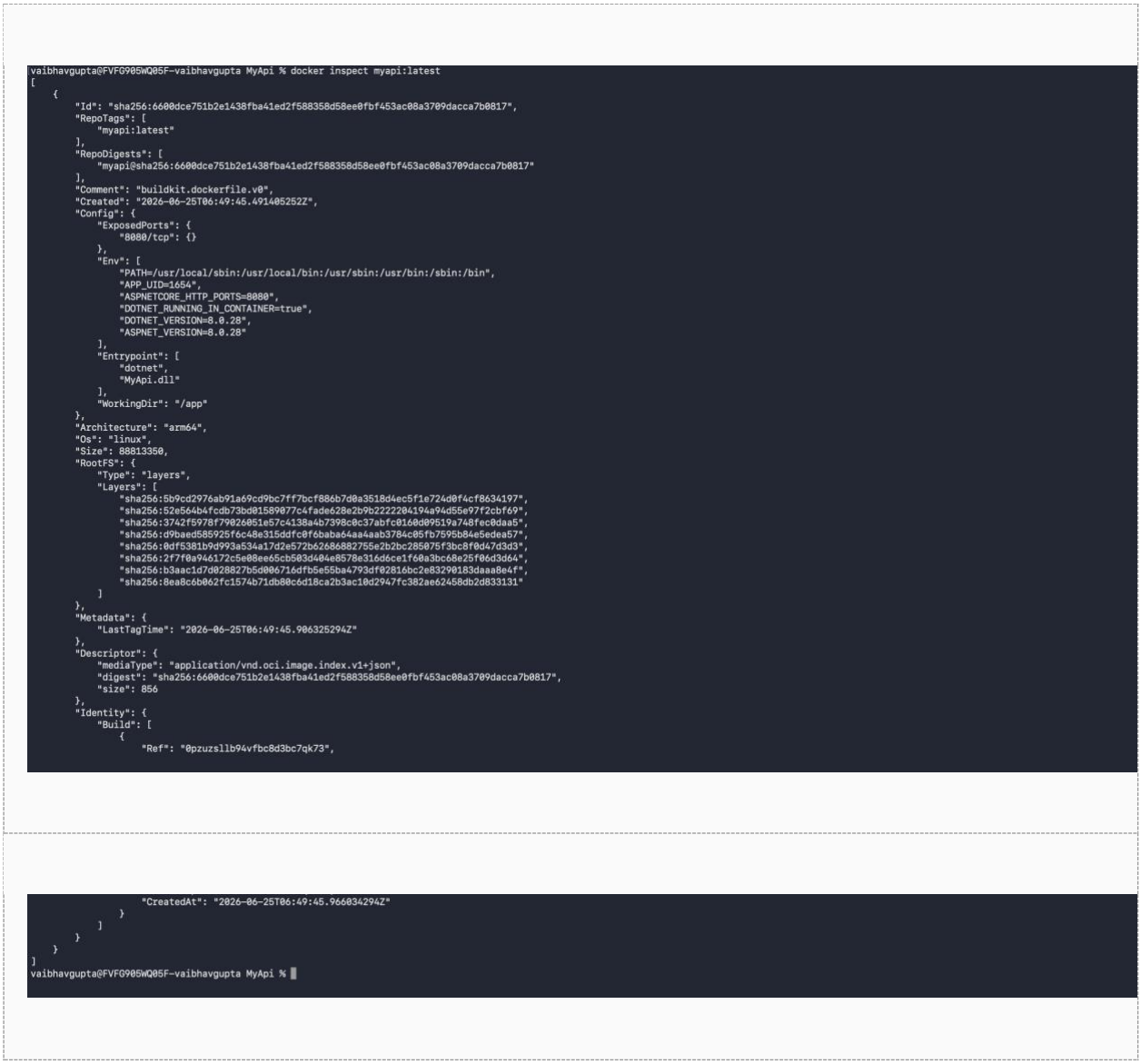
Step B2 — Confirm it is the correct image

This check matters: container tooling built into some IDEs produces debug-only images with a different startup command. Confirm yours was built cleanly from the Dockerfile.

```
docker inspect myapi:latest
```

What to check in the output

"Entrypoint" should read ["dotnet", "MyApi.dll"]
"Cmd" should NOT read ["bash"]
"Labels" should be empty, or absent entirely

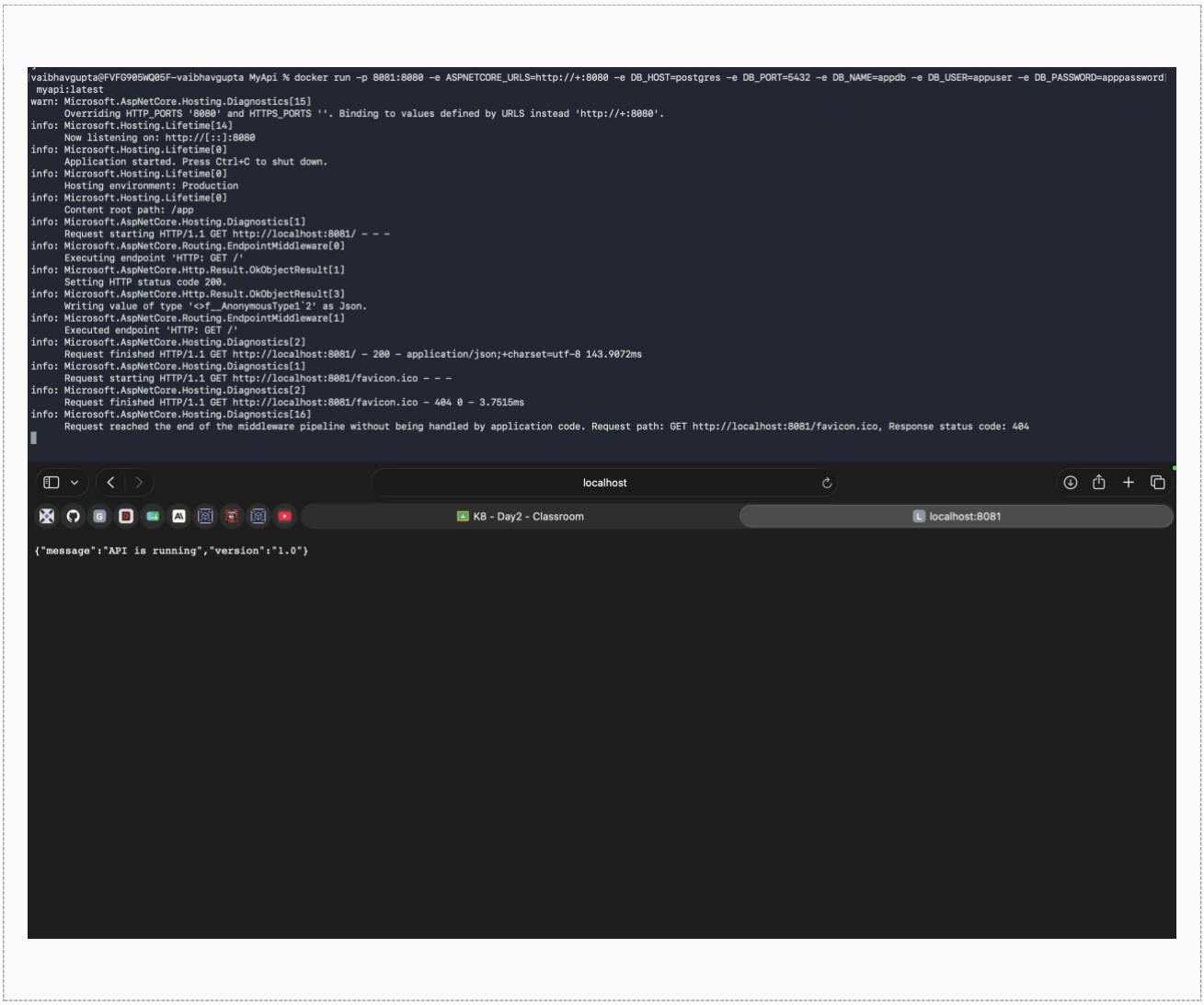


Step B3 — Smoke-test the image locally

This confirms the container starts and serves HTTP, before we ever involve Kubernetes. Pick a host port that is not already in use on your machine — 8081 is used here as an example.

```
docker run -p 8081:8080 -e ASPNETCORE_URLS=http://+:8080 -e DB_HOST=postgres -e
DB_PORT=5432 -e DB_NAME=appdb -e DB_USER=appuser -e DB_PASSWORD=apppassword
myapi:latest
```

Open http://localhost:8081/ in a browser. You should see a small JSON response confirming the API is running. The /db-check endpoint will fail at this point, since there is no host named postgres reachable from plain Docker — that is expected here, and gets resolved once we are inside the cluster.



Stop the container with Ctrl+C once confirmed.

Step B4 — Make the image visible to your cluster (kind only)

> **Skip this step if you chose Kubeadm**
>
> This step only applies if Docker Desktop's Kubernetes was set up using the kind engine. If you chose Kubeadm instead, your cluster shares the host's Docker image cache directly, and this step is unnecessary — continue straight to Part C.

A kind cluster runs its node as a separate container with its own isolated container runtime and image storage, entirely separate from your host machine's Docker image cache. Building an image with docker build on your host does not automatically make it visible inside the kind node. Skipping this step is the single most common cause of an ImagePullBackOff or ErrImageNeverPull error later, even though docker images on your host clearly shows the image present.

First, find the exact name of your node container:

```
docker ps
```

Look for a container whose name matches your cluster's node, typically ending in -control-plane (for example, desktop-control-plane). Note this exact name.

Then export the image from your host's Docker and import it directly into that node's container runtime (containerd, not Docker, is what the node actually uses internally):

```
docker save myapi:latest | docker exec -i desktop-control-plane ctr -n=k8s.io images import -
```

Replace desktop-control-plane with your actual node container name if different. This command pipes a tar export of your image directly into the node container, where containerd's import command registers it under the k8s.io namespace — exactly where Kubernetes looks for images on that node.

SCREENSHOT

Terminal output confirming the image import completed

Repeat this step any time you rebuild the image with a new docker build — the kind node will not automatically pick up the change otherwise.

Part C — Kubernetes Manifests

Create a new folder for your manifests, separate from the MyApi source folder, e.g. C:\k8s-training\manifests. Create each file below inside it.

Step C1 — ConfigMap (non-sensitive configuration)

This holds the database hostname (the Kubernetes Service name, not an IP address), port, database name, username, and API port. None of this is sensitive.

02-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_HOST: "postgres"
  DB_PORT: "5432"
  DB_NAME: "appdb"
  DB_USER: "appuser"
  API_PORT: "8080"
  ASPNETCORE_URLS: "http://+:8080"
```

Step C2 — Secret (sensitive configuration)

This holds only the database password. Separating it from the ConfigMap is intentional: secrets get different handling in real clusters (encryption at rest, restricted access via RBAC), and keeping it isolated here makes that boundary explicit.

02-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
type: Opaque
stringData:
  DB_PASSWORD: "apppassword"
```

Note

This password is in plain text here purely for learning purposes. In a real environment this value would come from a secrets manager or vault, never typed directly into a file that gets committed to source control.

Step C3 — PostgreSQL Deployment and Service

Notice the password is no longer hardcoded — it is pulled from the Secret created in Step C2 via secretKeyRef.

02-postgres.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
  labels:
    app: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              value: appdb
            - name: POSTGRES_USER
              value: appuser
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: DB_PASSWORD
---
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

Step C4 — API Deployment and Service

This is the most detailed manifest in this lab. It injects the entire ConfigMap as environment variables, injects the password individually from the Secret, and defines both a liveness probe (restart the pod if this fails) and a readiness probe (stop sending traffic to the pod if this fails, without restarting it).

02-api.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  labels:
    app: api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: myapi:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 8080
              name: http
          envFrom:
            - configMapRef:
                name: app-config
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: DB_PASSWORD
          livenessProbe:
            httpGet:
              path: /
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 10
            timeoutSeconds: 5
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
            timeoutSeconds: 3
            failureThreshold: 2
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
```

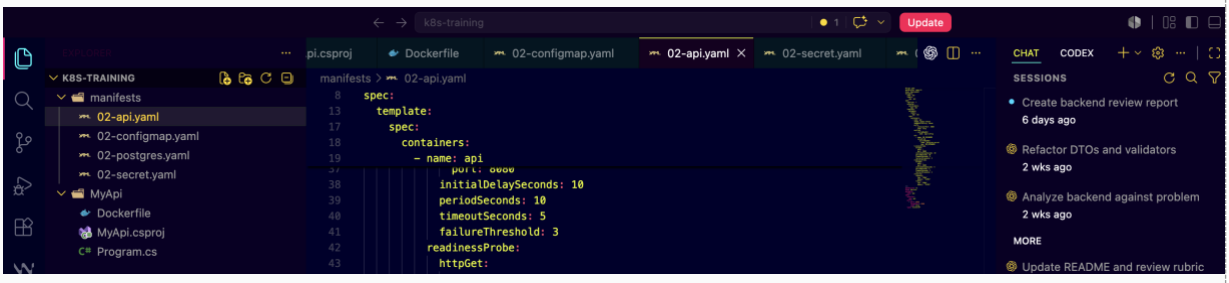
```
---
apiVersion: v1
kind: Service
metadata:
  name: api
spec:
  selector:
    app: api
  ports:
    - port: 8080
      targetPort: 8080
      name: http
  type: ClusterIP
```

Why imagePullPolicy: Never is set

Kubernetes' default behavior for an image tagged :latest is to always attempt a pull from a remote registry, even if a matching image already exists locally. Since myapi was built and loaded locally rather than published anywhere, that default would cause every deployment to fail with ImagePullBackOff. Setting this explicitly to Never tells Kubernetes to use only what is already present on the node — exactly the image loaded in Step B4.

Confirm your manifests folder now contains exactly these four files:

```
C:\k8s-training\manifests\
  |-- 02-configmap.yaml
  |-- 02-secret.yaml
  |-- 02-postgres.yaml
  \-- 02-api.yaml
```



Part D — Deploy to Kubernetes

Step D1 — Confirm a clean cluster

```
kubectl get pods
```

Expect no application pods. If anything unexpected remains from an earlier lab, delete it before continuing.

Step D2 — Apply config and secret first

These must exist before the Deployments that reference them, otherwise the referencing Pods will fail to start.

```
kubectl apply -f 02-configmap.yaml
kubectl apply -f 02-secret.yaml
```

Step D3 — Deploy PostgreSQL and wait for it to be ready

```
kubectl apply -f 02-postgres.yaml
kubectl get pods --watch
```

Wait until the postgres pod shows 1/1 under READY and Running under STATUS, then press Ctrl+C.

Step D4 — Deploy the API

```
kubectl apply -f 02-api.yaml
kubectl get pods --watch
```

Wait until the api pod also shows 1/1 Running, then press Ctrl+C.

```
vaibhavgupta@FVFG905WQ05F-vaibhavgupta manifests % kubectl apply -f 02-api.yaml
deployment.apps/api created
service/api created
vaibhavgupta@FVFG905WQ05F-vaibhavgupta manifests % kubectl get pods --watch
NAME                        READY   STATUS             RESTARTS   AGE
api-65c4f494df-rtq48        0/1     ErrImageNeverPull  0          17s
my-app-59854d5646-zc8jc     1/1     Running            0          17m
postgres-5f64456b7d-p7dvj   1/1     Running            0          3m15s
my-app-59854d5646-zc8jc     1/1     Running            0          18m
```

Step D5 — Verify all created objects

```
kubectl get deployments
kubectl get services
kubectl get configmaps
kubectl get secrets
```

```
vaibhavgupta@FVFG905WQ05F-vaibhavgupta manifests % kubectl get deployments
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
api        0/1     1            0           80s
my-app     1/1     1            1           18m
postgres   1/1     1            1           4m18s
vaibhavgupta@FVFG905WQ05F-vaibhavgupta manifests % kubectl get services
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
api          ClusterIP   10.96.113.88    <none>        8080/TCP   86s
kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP    20m
postgres     ClusterIP   10.96.111.220   <none>        5432/TCP   4m24s
vaibhavgupta@FVFG905WQ05F-vaibhavgupta manifests % kubectl get configmaps
NAME               DATA   AGE
app-config         6      4m41s
kube-root-ca.crt   1      20m
vaibhavgupta@FVFG905WQ05F-vaibhavgupta manifests % kubectl get secrets
NAME              TYPE     DATA   AGE
postgres-secret   Opaque   1      4m49s
vaibhavgupta@FVFG905WQ05F-vaibhavgupta manifests %
```

Part E — Prove Service Discovery Works

This is the real test of this lab: the API container, running in its own Pod, must successfully reach PostgreSQL using only the Service name 'postgres' — not an IP address.

Step E1 — Find the API pod's name

```
kubectl get pods
```

Step E2 — Open a shell inside the API pod

```
kubectl exec -it my-app-59854d5646-zc8jc -- /bin/bash
```

Step E3 — From inside the pod, confirm DNS resolves the Service name

```
getent hosts postgres
```

This should return an internal cluster IP address next to the name postgres, proving Kubernetes' internal DNS resolved the Service name automatically.

Exit the pod shell with the exit command before continuing.

Step E4 — Hit the API's own /db-check endpoint

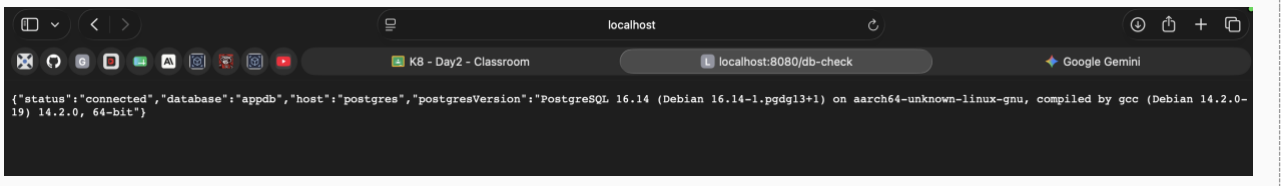
This is the most direct proof available. The /db-check endpoint inside your API code opens a real connection to PostgreSQL using the DB_HOST value injected from the ConfigMap, which is simply the string "postgres".

```
kubectl port-forward service/api 8080:8080
```

Leave that running, then in a separate terminal or browser tab, open:

```
http://localhost:8080/db-check
```

A successful response will include status: connected along with the PostgreSQL version string, confirming the full chain: ConfigMap and Secret injection, DNS-based service discovery, and a live database connection, all working together.



Troubleshooting Reference

Symptom	Likely Cause / Fix
Pod stuck in ContainerCreating	Image is still downloading. Wait, or run kubectl describe pod <name> and check the Events section.

Pod shows CrashLoopBackOff	The container is starting and immediately exiting. Run kubectl logs <pod-name> to see the application's own error output.
API pod running but /db-check fails	Confirm the postgres pod is Running first, and that the ConfigMap value DB_HOST matches the Service name exactly.
docker inspect shows Cmd: ["bash"]	You built or ran an IDE-generated debug image instead of the image from this lab's Dockerfile. Rebuild with docker build -t myapi:latest . from a plain terminal.
"port is already allocated" when running locally	Another container or process already owns that host port. Pick a different host port, e.g. -p 8082:8080.
Pod shows ImagePullBackOff	If using kind, the image was built on the host but never loaded into the kind node's separate image store. Repeat Step B4.
Pod shows ErrImageNeverPull	imagePullPolicy is set to Never, but the image genuinely is not present inside the kind node yet. Repeat Step B4, then delete the pod so a fresh one is scheduled.

Key takeaway

Your API never knew an IP address for PostgreSQL at any point. It only ever knew the name "postgres" — injected from a ConfigMap, resolved by Kubernetes' internal DNS. If the postgres Pod is deleted and recreated with a new IP tomorrow, nothing in your API or its configuration needs to change.